

Zackery Fite

CS-300

Southern New Hampshire University

Professor Phillip Grabowski

February 20<sup>th</sup>, 2026

### **CS-300: Project One**

The academic advisors in the Computer Science department at ABCU have expressed satisfaction with the progress made on the current system. As a result, they are now requesting a comprehensive evaluation of several data structure approaches for storing and managing course information. The goal of this evaluation is to determine which structure would provide the most efficient and scalable performance for a new academic planning application currently under development. The application will include:

- The system must **read and parse** course **data from a CSV file**, where each row represents **a single course**, **create course objects** containing the course number, title, and prerequisites, and store those objects in an appropriate data structure.
- **Create course objects to** indicate each course, in which the course number will be represented by **an alphanumeric string**.
- **Store each object in a data structure.**
- **Search, retrieve, and display** an individual **course object using the** course number.
- Display all courses in a sorted format by course number.

This task includes evaluating **the time** requirements **and complexities of the application based on the three** different data structures we have been working with: **vector, hash table,**

**and binary search tree.** Then, we have **been asked to give a recommendation based on the analysis.** Each of these structures **is unique** and comes with its own set of advantages and disadvantages, which we will point out.

Vectors store elements by index in contiguous memory, which allows for more efficient memory management and fast insertions at the end. However, since they are typically unsorted, searching for elements can be slow unless it's accessing by index, and sorting performance depends on the algorithm used. Vectors are a good choice when multiple elements need to be added, and sorting is infrequent.

**Hash tables store data in buckets using a hashed key**, enabling highly efficient data storage and retrieval. While they provide fast insertion and search operations, the data is not maintained in sorted order, and sorting cannot be performed in place without additional processing. As a result, operations requiring ordered data can be computationally expensive. Overall, hash tables are a strong choice when fast lookups and insertions are the primary requirements.

Binary Search Trees offer reliable performance for insertions and searches, though hash tables often provide faster lookups. Their primary advantage is maintaining data in sorted order, eliminating the need for additional sorting. This makes them especially useful when ordered data is required. Additionally, BSTs support efficient in-order traversal, which allows data to be processed sequentially from smallest to largest, and they can efficiently handle range-based queries compared to hash tables.

These are the methods that will be evaluated for each of the data structures.

### **FUNCTION DisplayMenu()**

// The **method remains the same for all** the data structures

// Time Complexity =  $O(1)$

// Space Complexity =  $O(1)$

Print the following menu options:

**1. Load Data**

**2. Display Course**

**3. Display Course Catalog**

**4. Exit**

Read the user **input into selection**

### **SWITCH selection**

IF selection equals 1

Call LoadData(ParseCSVFile(fileLocation))

ELSE IF selection equals 2

Print message requesting Course Number

Read input into courseNumber

    Call DisplayCourse(courseNumber)

ELSE IF selection equals 3

Call DisplaySortedCourseList()

ELSE IF selection equals 4

Call exit()

END SWITCH

END FUNCTION

### **FUNCTION LoadData (parsedFile)**

//Time Complexity:  $O(n)$

//Space Complexity  $O(n)$

Initialize data structure courses

FOR each row in parsed file

    Create a new Course Object

Set course.courseNumber equal to first element of row  
Set course.courseName equal to second element of row

Initialize empty list prerequisites

FOR each element in row starting at index 2  
ADD element to prerequisites  
END FOR

Set course.prerequisites equal to prerequisites  
Call courses.Insert(Course)

END FOR  
END FUNCTION

### **FUNCTION DisplayCourse(courseNumber)**

//Time Complexity: Depends on the data structure  
//Space Complexity: O(1)

Set course equal to Search(courseNumber)

IF course exists  
Call DisplayCourseDetails(course)  
ELSE  
Print "Course not found."  
END IF  
END FUNCTION

### **FUNCTION DisplaySortedCourseList()**

// Time Complexity: Depends on the data structure  
// Space Complexity: O(n)

Set coursesList equal to SortCourses()

FOR each course in coursesList  
Call DisplayCourseDetails(course)  
END FOR  
END FUNCTION

### **FUNCTION Exit()**

// Time Complexity: O(1)  
// Space Complexity: O(1)

Return 0 to main function

END FUNCTION



**FUNCTION ParseCSVFile(fileLocation)**

// Time Complexity:  $O(n)$

// Space Complexity:  $O(n)$

Open file stream using fileLocation

IF file fails to open

Return error

END IF

Initialize empty list inputLines

WHILE file has more lines

Read line

Add line to inputLines

END WHILE

Initialize empty list rows

Parse first line as header

Split header by comma

Store header elements

Add header to rows

FOR each line in inputLines starting at second line

Split line by comma into rowElements

IF number of elements in rowElements  $< 2$

Return error

END IF

ADD rowElements to rows

END FOR

Initialize empty list courseNumbers

FOR each row in rows starting at second row

Add first element (course number) to courseNumbers

END FOR

FOR each row in rows starting at second row

FOR each element starting at index 2

IF element is not in courseNumbers

Return error  
END IF  
END FOR

END FOR

Return rows

END FUNCTION

**FUNCTION DisplayCourseDetails(course)**

// Time Complexity:  $O(n)$  relative to prerequisites

// Space Complexity:  $O(1)$

Print course.courseNumber

Print course.courseName

IF course.prerequisites is not empty

Print "Prerequisites."

FOR each prerequisite in course.prerequisites

Print prerequisite

END FOR

ELSE

Print "No Prerequisites"

END IF

END FUNCTION

**FUNCTION Insert(course)****Vector Implementation**

// Time Complexity: Average  $O(1)$ , Worst  $O(n)$

Appended course to vector

**Hash Table Implementation**

// Time complexity: Average  $O(1)$ , Worst  $O(n)$

Compute key using hash(course.courseNumber)

IF table at key is empty

Create new linked list

Insert course

ELSE

Append course to linked list at key

END IF

**Bianary Search Tree Implementation**

// Time Complexity: Average:  $O(\log n)$ , Worst  $O(n)$

IF root is null

Set root equal to new node containing course

ELSE

Call AddNode(root, course)

END IF

END FUNCTION



**FUNCTION AddNode(node, course)**

IF course.courseNumber is less than node.course.courseNumber

IF node.left is null

Set node.left equal to new node containing course

ELSE

Call AddNode(node.left, course)

END IF

ELSE

IF node.right is null

set node.right equal to new node containing course

ELSE

Call AddNode(node.right, course)

END IF

END IF

END FUNCTION

**FUNCTION Search(courseNumber)**

Vector Implementation

// Time Complexity:  $O(n)$

FOR each course in vector

IF course.courseNumber equals courseNumber

Return Course

END FOR

Return empty course

Hash Table Implementation

// Time Complexity: Average  $O(1)$ , Worst  $O(n)$

Compute key using hash(courseNumber)

Set list equal to table at key

IF list is empty

Return empty course

END IF

FOR each node in list

IF node.course.courseNumber equals courseNumber

Return node.course

END FOR

Return empty course



### Binary Search Tree Implementation

// Time Complexity: Average  $O(\log n)$ , Worst  $O(n)$

Set current equal to root

WHILE current is not null

IF current.course.courseNumber equals courseNumber

Return current.course

ELSE IF courseNumber is less than current.course.courseNumber

Set current equal to current.left

ELSE

Set current equal to current.right

END WHILE

Return empty Course

END FUNCTION

### Analysis Chart:

#### Applications independent of structure implementation

| Average Case     | Display Menu() | Parse() | Display course() | Display catalog() | Exit() | Total  |
|------------------|----------------|---------|------------------|-------------------|--------|--------|
| Time Complexity  | 4              | $O(n)$  | $O(n)$           | $O(n)$            | 1      | $O(n)$ |
| Space Complexity | 4              | $O(n)$  | 2                | 3                 | 1      | $O(n)$ |

#### Applications dependent on data structure implementation

| Average Case       | Load()      | Search()    | Sort()        | Total       |
|--------------------|-------------|-------------|---------------|-------------|
| Time Complexity    |             |             |               |             |
| Vector             | $O(1)$      | $O(n)$      | $O(n*\log n)$ | $O(n)$      |
| Hash Table         | $O(1)$      | $O(1)$      | $O(n^2)$      | $O(n^2)$    |
| Binary Search Tree | $O(\log n)$ | $O(\log n)$ | 1             | $O(\log n)$ |
| Space Complexity   |             |             |               |             |
| Vector             | $O(1)$      | $O(1)$      | $O(\log n)$   | $O(\log n)$ |
| Hash Table         | $O(n)$      | $O(n)$      | $O(n+\log n)$ | $O(n)$      |
| Binary Search Tree | $O(n)$      | $O(n)$      | 1             | $O(n)$      |

### Recommendation

Based on the performance analysis, I recommend using the binary search tree as the most efficient overall data structure for this program. While vectors are simple to implement, they require linear time to search for a specific course, which becomes increasingly inefficient as the number of courses grows. Hash tables generally improve search performance; however, they introduce additional overhead, particularly when handling collisions and when sorted output is

required. Since hash tables do not naturally maintain order, extra processing is needed to sort the data before displaying a course catalog in order.

In contrast, a binary search tree provides a strong balance between efficiency and functionality. It offers average-case logarithmic time complexity for insertion and search operations, while also maintaining elements in sorted order through an in-order traversal. This eliminates the need for additional sorting steps when displaying the full course catalog. Due to this combination of efficient searching and built-in ordering, **the binary search tree performs significantly better** overall than both the vector and hash table implementations.